FACULDADE DE TECNOLOGIA DE SÃO PAULO

MAICON ROBERTO NASCIMENTO GAMBINI JUNIOR

Sistemas de Recomendação de Tarefas Aplicados a Gestão de Equipes e Produtividade

SÃO PAULO

2025

FACULDADE DE TECNOLOGIA DE SÃO PAULO

MAICON ROBERTO NASCIMENTO GAMBINI JUNIOR

SISTEMAS DE RECOMENDAÇÃO DE TAREFAS APLICADOS A GESTÃO DE EQUIPES E PRODUTIVIDADE

Trabalho submetido como exigência parcial

para a obtenção do Grau de Tecnólogo em

Análise e Desenvolvimento de Sistemas

Orientador: Professora Grace Anne Pontes

Borges

SÃO PAULO

2025

FACULDADE DE TECNOLOGIA DE SÃO PAULO

MAICON ROBERTO NASCIMENTO GAMBINI JUNIOR

SISTEMAS DE RECOMENDAÇÃO DE TAREFAS APLICADOS A GESTÃO DE EQUIPES E PRODUTIVIDADE

Trabalho submetido como exigência parcial para a obtenção do Grau de

Tecnólogo em Análise e Desenvolvimento de Sistemas.

Parecer do Professor Orientador

_____

_____

_____

Conceito/Nota Final: _____

Atesto o conteúdo contido na postagem do ambiente TEAMS pelo aluno e assinada por mim para avaliação do TCC.

Orientador: Profa. Ma. Grace Anne Pontes Borges

SÃO PAULO, ____ de _____ de 2025.

Assinatura do Orientador

Assinatura do aluno

AGRADECIMENTOS

Agradeço a minha mãe e meu pai que me incentivaram a concluir esse trabalho durante os momentos difíceis. Aos amigos que me apoiaram e incentivaram a continuar com a faculdade e o tema desse TCC. Agradeço a professora Grace Borges, orientadora, pelas correções, acompanhamento e suporte no desenvolvimento desse trabalho

RESUMO

O presente trabalho tem como tema principal a aplicação de Sistemas de Recomendação e técnicas de Inteligência Artificial para apoiar a gestão de tarefas em equipes, com foco em produtividade e organização do trabalho. A pesquisa investiga o estudo dos fundamentos de sistemas de recomendação e aprendizado de máquina e com a análise de casos de referência da indústria, como *Netflix* e *Amazon*, a fim de identificar práticas, arquiteturas e métricas relevantes para o problema de alocação de atividades. A partir dessas ideias e pesquisas, foi proposto um modelo conceitual de sistema de recomendação de tarefas voltado à gestão de equipes e desenvolvida utilizando a plataforma *n8n* como arquitetura, *PostgreSQL* como banco vetorial, linguagem de programação e agentes de IA para ranqueamento e explicação das sugestões. Foram realizados testes funcionais em dois fluxos principais – priorização de tarefas por colaborador e recomendação do melhor colaborador para uma tarefa – que demonstraram a viabilidade da abordagem, com geração de listas ordenadas, scores e justificativas coerentes com o contexto. As conclusões do estudo apontaram que a solução da proposta é tecnicamente viável e apresenta potencial para reduzir o esforço manual de alocação, embora ainda dependa de avaliações em ambiente real e de métricas quantitativas de produtividade para comprovar seu impacto prático.

Palavras-chave: Sistemas de Recomendação, Gestão de Tarefas, Produtividade, Gestão de Equipes, Inteligência Artificial

ABSTRACT

This study addresses the application of Recommender Systems and Artificial Intelligence techniques to support task management in teams, with a focus on productivity and work organization. The research presents the foundations of recommender systems and machine learning and analyses industry case studies, such as Netflix and Amazon, in order to identify practices, architectures and metrics relevant to the task allocation problem. Based on these ideas and findings, a conceptual model of a task recommendation system for team management was proposed and implemented as a proof of concept using the n8n platform as the orchestration layer, PostgreSQL with a vector extension as the vector database, and AI agents for ranking and explaining the recommendations. Functional tests were carried out on two main flows – task prioritization for a specific team member and recommendation of the most suitable team member for a given task – which demonstrated the feasibility of the approach by generating ordered lists, scores and justifications consistent with the context. The conclusions of the study indicate that the proposed solution is technically viable and has the potential to reduce the manual effort involved in task allocation, although further evaluation in real-world environments and quantitative productivity metrics are still required to confirm its practical impact.

***Keywords**: Recommendation Systems, Task Management, Productivity, Team Management, Artificial Intelligence*

LISTA DE FIGURAS

Pág.

Figura 1 - Primeiro fluxo no n8n.....31
Figura 2 - Exemplo de saída http usando Postman 32
Figura 3 - Exemplo de http dentro do n8n 32
Figura 4 - Nós de código Javascript dentro do n8n 33
Figura 5 - Nós de Agente de IA e banco vetorial no n8n 34
Figura 6 - Nós de Merge e Código Javascript do n8n.....34
Figura 7 - Arquitetura do segundo fluxo no n8n.....36
Figura 8 - Nós de chat trigger + Agente IA **n8n** 37
Figura 9 - Nós de Código Javascript segundo fluxo n8n 37
Figura 10 - Nós de Agente de IA + PostgreSQL segundo fluxo n8n.....38
Figura 11 - Exemplo de saída teste CT1 n8n 40
Figura 12 - Exemplo de saída teste CT2 n8n 42
Figura 13 - Exemplo de saída teste CT3 n8n 43
Figura 14 - Exemplo de saída teste CT4 n8n 43
Figura 15 - Exemplo de saida teste CT5 n8n 44
Figura 16 - Exemplo da saída teste CT6 n8n 45

SUMÁRIO

Pág.

1. INTRODUÇÃO 9
1.1 Problema 9
1.2 Hipóteses 10
1.3 Objetivo 11
1.4 Justificativa 12
2. FUNDAMENTOS DE SISTEMA DE RECOMENDAÇÃO 13
2.1 Sistema de Recomendação 13
2.1.1 Definições e Arquitetura 13
2.1.2 Tipos de Sistemas de Recomendação 14
2.1.3 Métricas e Abordagens de Avaliação 15
2.2 Aprendizado de Máquina 16
2.2.1 Definição e Modalidades 16
3. CONTEXTO DE APLICAÇÃO DE SOFTWARE: PRODUTIVIDADE, GESTÃO DE EQUIPES E GESTÃO DE PROJETOS 19
3.1 Produtividade e Software 19
3.2 Gestão de Equipe e Gestão de Projetos 20
4. ESTUDO DE CASO DE SISTEMAS DE RECOMENDAÇÕES 22
4.1 Estudo de caso: Netflix 22
4.2 Estudo de caso: Amazon 23
5. PROPOSTA DE SISTEMA DE RECOMENDAÇÃO PARA GESTÃO DE TAREFAS 26
5.1 MODELO CONCEITUAL 26
5.1.1 Impacto esperado na produtividade 27
5.2 DESENVOLVIMENTO DA SOLUÇÃO 29
5.2.1 Infraestrutura do Sistema de Recomendação: n8n e PostgreSQL 29
5.2.2 Arquitetura do Sistema de Recomendação 29
5.3 Testes do Sistema de Recomendação 39
5.3.1 Casos de Teste – Fluxo 1 39
5.3.2 Casos de Teste – Fluxo 2 42
5.4 Análise da Solução 45
6. CONCLUSÃO 47
REFERÊNCIAS 49

1. INTRODUÇÃO

No cenário dinâmico e complexo do trabalho contemporâneo, a gestão eficiente de equipes e a produtividade das pessoas tornaram-se pilares fundamentais para o sucesso organizacional. Em ambientes cada vez mais digitais, gestores precisam distribuir tarefas entre profissionais com diferentes competências, níveis de experiência e cargas de trabalho, ao mesmo tempo em que lidam com prazos apertados e demandas que mudam rapidamente.

Atualmente, a área de Tecnologia tem avançado no desenvolvimento de técnicas capazes de apoiar a tomada de decisão em cenários marcados por grande volume de informação. Entre essas técnicas, destacam-se a Inteligência Artificial e, em particular, os Sistemas de Recomendação, que já se consolidaram como componentes centrais em domínios como comércio eletrônico, entretenimento e mídia digital. Esses sistemas auxiliam os usuários a encontrar itens relevantes em catálogos extensos, oferecendo listas personalizadas de produtos, filmes, músicas ou notícias (Jiang et al 2018).

A experiência acumulada em aplicações industriais mostra que Sistemas de Recomendação podem aumentar receita e satisfação do usuário ao reduzir a sobrecarga de informação e tornar as escolhas mais eficientes (Pu et al 2011).

Esse histórico bem-sucedido em outros domínios torna a utilização de Sistemas de Recomendação promissora também no contexto de gestão de equipes e tarefas: em vez de recomendar filmes ou produtos, o sistema pode recomendar quem deve executar determinada atividade, com base em habilidades, histórico, disponibilidade e desempenho (Jiang et al. 2018).

1.1 Problema

Em ambientes de trabalho dinâmicos com múltiplas tarefas e equipes diversificadas, gestores enfrentam um desafio complexo: distribuir eficientemente as atividades entre membros da equipe que possuem diferentes habilidades, competências e níveis de disponibilidade. Alocar cada tarefa ao colaborador mais adequado exige avaliar minuciosamente a especialização de cada um e sua carga de trabalho atual, tornando o processo trabalhoso e propenso a falhas conforme aumentam a quantidade de tarefas e o tamanho da equipe. (Antonio, 2009).

Se faz necessário verificar essa promessa de otimização que o Sistema de Recomendação representa e investigar a fundo sua aplicabilidade nesse sentido. Sendo assim, surgem alguns questionamentos essenciais para o desenvolvimento desse trabalho:

- **Q1:** Como um Sistema de Recomendação pode, de fato, aprimorar a alocação de tarefas, correlacionando as demandas de atividade com as competências e a carga de trabalho dos colaboradores, agregando valor no tempo de gestores e reduzindo o retrabalho?
- **Q2:** Quais os mecanismos necessários para que um Sistema de Recomendação de Tarefas possa aprender e evoluir com o histórico e dados da equipe, tornando suas sugestões progressivamente mais precisas e eficazes?
- **Q3:** Qual o impacto potencial da implementação de um Sistema de Recomendação de Tarefas na redução de gargalos, melhoria de comunicação e na produtividade coletiva de uma equipe, visando um ecossistema de trabalho mais eficiente e sustentável?

1.2 Hipóteses

Considerando a complexidade inerente à gestão de equipes no cenário atual, onde a alocação manual de tarefas e a falta de visibilidade sobre as capacidades individuais são obstáculos significativos, as seguintes proposições guiarão esta pesquisa:

• H1 – Alocação orientada por recomendação:

A alocação de tarefas orientada por um Sistema de Recomendação, que correlaciona a demanda da atividade com as competências específicas e a carga de trabalho de cada colaborador, **tende a resultar** em maior qualidade percebida nas entregas e menor índice de retrabalho, quando comparada à distribuição tradicional baseada apenas na intuição do gestor.

• H2 – Aprendizado com o histórico da equipe

Um Sistema de Recomendação que incorpora dados históricos da equipe (por exemplo, tempo de conclusão por tipo de tarefa, taxas de retrabalho e desempenho

por colaborador) **pode se tornar progressivamente mais preciso e eficaz** em suas sugestões, adaptando-se de forma dinâmica ao ritmo, às competências e à evolução do time ao longo do tempo.

• H3 – Gestão de tempo do gestor:

A automação parcial da alocação de tarefas operacionais por meio de um Sistema de Recomendação **contribui para liberar** o tempo do gestor, permitindo maior foco em atividades estratégicas, como planejamento e mentoria, e favorecendo um melhor alinhamento da equipe com os objetivos de negócio da organização.

1.3 Objetivo

Diante dos desafios persistentes na gestão de equipes e na busca por maior produtividade no ambiente corporativo, o objetivo geral deste trabalho é:

Propor um modelo de Sistema de Recomendação aplicado a alocação de tarefas em equipes de trabalho, visando apoiar a tomada de decisão de gestores e aumento da produtividade.

Para alcançar esse objetivo geral, são definidos os objetivos:

- **OBJ1:** Estudar os fundamentos teóricos de Sistemas de Recomendação e Aprendizado de Máquina, com ênfase em aplicações que lidam com sobrecarga de informação e personalização de decisões (Riedl et al 1999) (Pu et al 2012).

- **OBJ2:** Investigar como Sistemas de Recomendação são utilizados em casos de sucesso da indústria, como *Netflix e Amazon*, identificando práticas, arquiteturas e métricas relevantes para a construção de modelos aplicados à gestão de tarefas (Jiang et al. 2018).

- **OBJ3:** Definir um modelo conceitual de Sistema de Recomendação voltado para a distribuição de tarefas em equipes, especificando fontes de dados, atributos de tarefas e colaboradores, bem como critérios de decisão e de avaliação de desempenho.

- **OBJ4:** Implementar uma prova de conceito do modelo proposto, integrando técnicas de Inteligência Artificial e banco de dados vetorial ao fluxo de trabalho de uma ferramenta de gestão de tarefas.

1.4 Justificativa

A justificativa para esta pesquisa emerge da urgência em transformar a gestão de equipes e a produtividade no cenário corporativo atual. Como detalhado na seção 1.1, a intervenção manual e constante dos gestores na alocação de tarefas ainda é uma realidade, evidenciando um "quebra-cabeça diário" que resulta em ineficiência, gargalos e comunicação deficiente. Esta lacuna entre o potencial tecnológico e as necessidades operacionais ressalta a importância de abordagens inovadoras e eficazes.

A literatura em Sistemas de Recomendação mostra que essas tecnologias se tornaram componentes essenciais em contextos com grande volume de opções, ajudando usuários a encontrar itens relevantes e reduzindo a sobrecarga de informação (Riedl et al 1999). Estudos também indicam que sistemas de recomendação bem projetados podem aumentar significativamente a satisfação, a confiança e a intenção de uso dos usuários ao oferecer recomendações personalizadas e transparentes (Pu et al., 2012; Knijnenburg et al., 2012).

Apesar desse avanço, ainda são relativamente poucos os estudos que exploram o uso de Sistemas de Recomendação especificamente para a alocação de tarefas em equipes de trabalho, integrando dados de habilidades, disponibilidade, histórico de desempenho e prioridades de negócio. Assim, este trabalho se justifica por:

- Propor a adaptação de conceitos consolidados de Sistemas de Recomendação para um contexto ainda pouco explorado: a gestão de equipes e a distribuição de tarefas.
- Contribuir com um modelo conceitual e uma prova de conceito que demonstrem a viabilidade de utilizar IA e recomendação como apoio à tomada de decisão de gestores, em vez de focar apenas em recomendações de consumo.

2. FUNDAMENTOS DE SISTEMA DE RECOMENDAÇÃO

Este capítulo tem como objetivo apresentar os conceitos fundamentais que serão abordados ao longo deste trabalho. Serão detalhados os pilares tecnológicos e de gestão que sustentam a proposta, fornecendo a base teórica necessária para o entendimento e desenvolvimento da solução.

2.1 Sistema de Recomendação

Os Sistemas de Recomendação são ferramentas essenciais que ajudam os usuários a lidar com a sobrecarga de informação, fornecendo uma "sub-lista personalizada de itens" adaptados às suas preferências, ao invés de apresentar um catálogo completo (Riedl et al. 1999).

Essa sobrecarga de informação é um problema crescente, tornando difícil para os usuários a busca por informações em grandes *websites* (Jiang et al. 2018).

2.1.1 Definições e Arquitetura

Formalmente, um Sistema de Recomendação opera sobre um conjunto de usuários, um conjunto de itens e um histórico de interações, onde cada interação representa uma avaliação de um usuário sobre um item. O problema da recomendação baseada em conteúdo, por exemplo, é definir uma relação de similaridade entre itens para gerar lista de recomendações personalizadas (Brunialti et al. 2015).

Uma arquitetura típica para ***SRbC*** (Sistema de Recomendação baseado em Conteúdo) é composta por três módulos principais:

- **Módulo de Análise e Representação de Conteúdo**: Recebe as descrições dos itens, pré-processa e cria uma representação estruturada para o aprendizado do perfil do usuário.
- **Módulo de Personalização**: Utiliza a representação do item e o histórico de interações do usuário para inferir suas preferências. As avaliações podem ser explícitas (usuário avalia um item) ou implícitas (usuário interage com um item, como ler uma notícia ou permanecer em uma página).
- **Módulo de Filtragem**: Gera e apresenta a lista de recomendações ao usuário, tipicamente ordenada por um *score* de relevância (Brunialti et al. 2015).

2.1.2 Tipos de Sistemas de Recomendação

Os sistemas de recomendação podem ser classificados de diversas formas, dependendo da maneira como implementam a recomendação:

Sistemas Baseados em Conteúdo (*Content-Based Recommendation Systems - SRbC*): Geram recomendações com base nas **características dos itens** e no **perfil do usuário**, filtrando itens semelhantes àqueles que o usuário preferiu no passado (Barth, 2010).

Sistemas baseados em características: Permitem que os usuários expressem suas preferências em características específicas do item, sendo adequados para produtos de alto envolvimento, como notebooks, carros ou câmeras digitais (Pu et al. 2011). Incluem:

Baseados em Casos (*Case-based systems*): Solucionam novos problemas utilizando um banco de dados de experiências passadas, onde os itens são representados como "casos", e as recomendações são geradas recuperando casos mais similares à consulta do usuário (Pu et al. 2011).

Baseados em Utilidade (*Utility-based systems*): Fazem sugestões calculando a utilidade de cada objeto para o usuário, que deve especificar preferências para um conjunto de atributos (Pu el al. 2011).

Baseados em Conhecimento (*Knowledge-based systems*): Possuem conhecimento funcional sobre como um item atende a uma necessidade do usuário, podendo inferir a relação entre necessidade e recomendação (Pu el al. 2011).

Baseados em Crítica (*Critiquing-based systems*): Simulam um vendedor, recomendando opções e solicitando *feedback* na forma de "críticas" (Jiang et al. 2018).

Sistemas de Filtragem Colaborativa (*Collaborative Filtering - CF*): Baseiam-se nas **opiniões de usuários com gostos semelhantes**, encontrando "vizinhos" para o usuário alvo e recomendando itens que esses vizinhos apreciaram, mas que o usuário alvo ainda não experimentou (Barth, 2010). A técnica é frequentemente

chamada de *"k-nearest-neighbor" ou "user-based*". Um exemplo é o sistema *MovieLens*, que coleta avaliações de filmes e as usa para encontrar pessoas com gostos similares (Riedl et al. 1999).

Sistemas Baseados em Conhecimento (*Knowledge-based)*: As sugestões são baseadas em inferências sobre as **necessidades e preferências dos usuários** (Riedl et al. 1999).

Sistemas Híbridos (*Hybrid Recommender Systems*): Combinam diferentes abordagens (e.g., filtragem colaborativa e baseada em conteúdo) para **superar as fraquezas individuais** de cada técnica e aprimorar a qualidade da recomendação (Riedl et al. 1999).

2.1.3 Métricas e Abordagens de Avaliação

A avaliação de sistemas de recomendação é uma etapa crucial que evoluiu ao longo do tempo. Tradicionalmente, o foco estava no desempenho de algoritmos, especialmente na acurácia objetiva de predição. No entanto, a literatura mais recente reconhece que a acurácia por si só não é suficiente para garantir a satisfação ou construir a lealdade do usuário. Questões como a efetividade na apresentação, a explicação das sugestões e a inspiração de confiança influenciam fortemente a percepção geral do usuário sobre o sistema (Pu et al. 2011).

Diante disso, as abordagens de avaliação podem ser divididas em duas categorias principais: as focadas em métricas quantitativas de desempenho e as centradas na experiência do usuário.

Métricas Quantitativas de Desempenho: Essas métricas medem a performance do algoritmo de forma objetiva, sendo úteis para comparar diferentes modelos em um ambiente *offline* (controlado).

- **Métricas de Acurácia Preditiva:** Buscam medir o quão próximas as previsões do sistema estão das avaliações reais dos usuários. A métrica mais comum é o ***Mean Absolute Error (MAE)***, que calcula o desvio médio entre as notas preditas e as notas verdadeiras. Um valor de *MAE* menor indica, numericamente, uma maior acurácia (Riedl et al. 1999). Contudo, é importante notar que a acurácia objetiva medida por métricas como o *MAE* pode não ter uma correlação direta com a acurácia percebida pelo usuário final.

- **Medidas de Similaridade:** Em vez de avaliar a recomendação final, algumas métricas são usadas internamente pelos algoritmos. Em sistemas de filtragem colaborativa, por exemplo, o **coeficiente de correlação de *Pearson (PCC*)** é amplamente utilizado para calcular a similaridade entre os perfis de avaliação de diferentes usuários, sendo fundamental para encontrar "vizinhos" com gostos parecidos (Jiang et al. 2018).

***Frameworks* de Avaliação Centrados no Usuário:** Reconhecendo as limitações das métricas puramente numéricas, a pesquisa evoluiu para modelos de avaliação que consideram a qualidade da experiência do usuário como um todo.

- **Modelos de Qualidade da Experiência:** *Frameworks* como o ***ResQue (Recommender systems' Quality of user experience)*** foram desenvolvidos para avaliar a qualidade percebida dos Sistemas de Recomendação sob a perspectiva do usuário. Essa abordagem é mais holística e analisa múltiplos fatores, como a usabilidade do sistema, a utilidade das recomendações, a qualidade da interface, a satisfação geral e como essas qualidades influenciam as intenções de uso futuro (como intenção de compra ou de retorno ao sistema) (Pu et al. 2011; Knijnenburg et al. 2012). O *ResQue*, por exemplo, foi desenvolvido com base em teorias consolidadas de aceitação de tecnologia, como o T*AM (Technology Acceptance Model)* (Pu et al. 2011).

2.2 Aprendizado de Máquina

Diante disso, Aprendizado de Máquina (AM) é um campo da Ciência da Computação que se caracteriza pelo desenvolvimento de técnicas que permitem aos *softwares* melhorararem seu desempenho em uma tarefa aprendendo através da experiência. Ele é uma área promissora para a aplicação em Sistemas de Recomendação, especialmente quando baseados em conteúdo textual (Brunialti et al. 2015).

2.2.1 Definição e Modalidades

A teoria de Aprendizado de Máquina (AM) baseia-se nos princípios do aprendizado indutivo, onde modelos são determinados a partir de um conjunto de dados ou representações de experiências, existem três modalidades principais de AM:

Aprendizado Supervisionado: Algoritmos ajustam parâmetros de um modelo a partir do erro medido entre as respostas obtidas e as respostas esperadas (rotuladas) (Riedl et al. 1999).

Aplicação em SR: Frequentemente usado no módulo de Personalização para construir classificadores que, dado um item, determinam se ele é adequado ou não para um usuário específico. Um classificador é treinado para cada usuário com base no histórico de interações, onde experiências positivas definem a classe "adequada" e negativas definem a classe "não adequada" (Brunialti et al. 2015).

Algoritmos Comuns: Modelos probabilísticos *(como Expectation-Maximization e Redes Bayesianas), Support Vector Machine (SVM),* classificador de *Rocchio, k-Nearest Neighbor (k-NN)* e Regressão Logística (RL) (Brunialti et al. 2015)

Aprendizado Não-Supervisionado: Os parâmetros de um modelo são ajustados com base na maximização de medidas de qualidade das respostas obtidas, sem a necessidade de dados previamente rotulados (Riedl et al. 1999).

Algoritmos Comuns: Determinação de variáveis latentes, *Latent Semantic Analysis (LSA), Latent Dirichlet Allocation (LDA),* otimização gradiente descendente, *random walks* e técnicas de agrupamento como *k-means* (Brunialti et al. 2015).

Aprendizado Semi-Supervisionado: Caracterizado pelo uso de algoritmos híbridos que combinam recursos de correção de erro (supervisionado) e maximização de medidas de qualidade (não-supervisionado). Nos estudos analisados por (Brunialti et al. 2015), o uso de Aprendizado de Máquina semi-supervisionado não foi expressivo em *SRbC* textual. Em Sistemas de Recomendação baseados em Conteúdo Textual, as técnicas de AM são aplicadas em diversas fases:

- **Análise e Representação de Conteúdo**: Os itens textuais não estruturados são pré-processados (e.g., uso de *Vector Space Model (VSM)*, extração de *n-gramas*, *stopwords*, *stemming*) e submetidos a técnicas de Aprendizado de Máquina não-supervisionado para gerar uma representação estruturada e compacta, frequentemente com redução de dimensionalidade (Riedl et al.1999).
- **Personalização**: Técnicas de Aprendizado de Máquina supervisionado são usadas para o aprendizado do perfil do usuário. O perfil pode ser enriquecido

com informações demográficas ou de redes sociais (Brunialti et al. 2015).

- **Recomendação (Filtragem)**: Em alguns casos, técnicas de AM (como k-NN) são usadas diretamente para identificar itens similares aos que o usuário acessou, gerando a lista de recomendação (Brunialti et al. 2015).

3. CONTEXTO DE APLICAÇÃO DE SOFTWARE: PRODUTIVIDADE, GESTÃO DE EQUIPES E GESTÃO DE PROJETOS

A compreensão de como os Sistemas de Recomendação e o Aprendizado de Máquina podem otimizar a interação humana com a tecnologia permite estender seus princípios para além do *e-commerce* e entretenimento, impactando positivamente o ambiente de trabalho e a colaboração (Antonio, 2009). Em contextos gerais de *e-commerce,* como será mais aprofundado no capítulo 4, o "usuário" costuma ser um indivíduo que consome recomendação de produtos ou conteúdo.

Já em um sistema de gestão de equipes e produtividades, como é o tema desse trabalho, existem pelo menos dois papéis destintos: o **gestor,** responsável por tomar decisões de alocação e priorização de tarefas e os **colaboradores,** que seriam os perfis alimentados no sistema com as informações sobre suas competências e histórico de atuação.

3.1 Produtividade e *Software*

Na literatura de Sistemas de Recomendação aplicada a *e-commerce***, a produtividade do** usuário é entendida como a eficiência e a facilidade com que ele consegue atingir seus objetivos. Eles são projetados para **reduzir o esforço e o tempo** que os usuários despendem na tomada de decisões (Knijnenburg et al. 2012).

No contexto específico desse trabalho, isso significaria criar um modelo onde o **gestor** seria apoiado por recomendações pra essa alocação de tarefas, com base no histórico dos **colaboradores,** onde eles se beneficiaram de uma distribuição de trabalho mais coerente com seus perfis.

• **Redução de Esforço e Eficiência**: A capacidade de um sistema em minimizar o esforço inicial de elicitação de preferências é crucial, pois os usuários tendem a preferir a economia de esforço imediata em detrimento de uma gratificação tardia de maior acurácia (Pu et al. 2011). Um sistema eficaz, com alta ***perceived system effectiveness*** (eficácia percebida do sistema), está correlacionado com a **redução da atividade de navegação** e o **aumento do consumo** de conteúdo pelos usuários significando que eles encontram o que desejam mais rapidamente e com menos exploração. A **facilidade de uso percebida** *(perceived ease of use*) e a redução do **esforço cognitivo** são fatores que influenciam diretamente a satisfação e a adoção de um sistema (Pu et al. 2011; Knijnenburg et al. 2012).

• **Qualidade da Decisão**: O Sistema de Recomendação atua como ferramentas de apoio à decisão, ajudando os usuários a gerenciar o excesso de informações e a tomar decisões de alta qualidade sob restrições de tempo e conhecimento limitado. Isso se traduz em maior **confiança na decisão** tomada e na **satisfação** com a escolha final (Paula, 2023).

Um *software* projetado com esses princípios em mente, utilizando sistemas de recomendação, poderia **aumentar a produtividade** de seus usuários ao otimizar a descoberta de informações ou recursos relevantes, minimizando distrações e o tempo gasto em buscas improdutivas. (Paula, 2023).

3.2 Gestão de Equipe e Gestão de Projetos

Motivação e Contribuição para Metas Compartilhadas

- Estudos em psicologia social aplicados a comunidades *online* (como o MovieLens) demonstraram que os usuários dos sistemas utilizados nas pesquisas são mais motivados a **contribuir com informações** (e.g., avaliações) quando estão **cientes de suas contribuições únicas** para a comunidade (Pu et al. 2011).

- A definição de metas específicas e desafiadoras para os usuários (tanto gestores quanto colaboradores nesse caso) também se mostrou eficaz em aumentar as contribuições no contexto de gestão de projetos ou equipes, esses achados sugerem que a transparência sobre o **impacto individual** de cada membro no projeto e a definição de **metas claras e coletivas** podem fomentar um maior engajamento e contribuição (Paula, 2023).

Controle do Usuário e Engajamento

- Permitir que os usuários(gestores) tenham **controle sobre o processo de elicitação de preferências** pode aumentar o seu senso de controle e a motivação para interagir e contribuir mais no futuro (Riedl et al. 1999).

- Em um *software* de gestão de equipe ou projeto, dar aos gestores maior autonomia e controle sobre suas ferramentas e processos, ou permitir que personalizem o ambiente de trabalho, pode levar a um maior sentimento de propriedade e engajamento. (Riedl et al. 1999).

Equilíbrio entre Esforço e Benefício:

- A gestão do **custo-benefício** percebido pelo gestor é fundamental para a adoção de tecnologia se o sistema exigir muito esforço inicial sem um benefício claro, a aceitação será menor (Riedl et al. 1999).

- Em projetos, isso implica que as ferramentas e processos implementados devem justificar o esforço de aprendizado e uso com **benefícios tangíveis e imediatos** para a equipe, evitando que a "fadiga" da ferramenta prejudique a produtividade.

Portanto, ao desenvolver um software para gestão de equipes que incorpore esses princípios de sistemas de recomendação tende a aumentar a produtividade de gestores e colaboradores ao apoiar a tomada de decisão e a organização do trabalho.

Para o **gestor**, o sistema reduz o esforço necessário para analisar histórico, competências e carga de trabalho antes de alocar uma tarefa. Para os **colaboradores**, contribui para que recebam atividades mais coerentes com seus perfis e prioridades, diminuindo o tempo gasto com incerteza, retrabalho e buscas improdutivas (Paula, 2023).

4. ESTUDO DE CASO DE SISTEMAS DE RECOMENDAÇÕES

Este capítulo sintetiza a metodologia adotada para analisar como *Netflix* e *Amazon* estruturam seus sistemas de recomendação, a *Netflix* destaca o papel de competições e métricas de engajamento, enquanto a *Amazon* exemplifica a eficácia da filtragem colaborativa baseada em itens.

As lições desses casos fundamentam a criação de um modelo conceitual de recomendação de tarefas voltado a *softwares* de produtividade, que será apresentado no capítulo 5. O modelo proposto define a coleta de dados, modelagem, algoritmos e critérios de avaliação para alocar tarefas de forma inteligente, o objetivo é mostrar que, ao adaptar práticas consolidadas, pode-se aumentar a eficiência e a satisfação das equipes (Amatriain et al. 2015; Smith et al. 2017).

4.1 Estudo de caso: Netflix

A *Netflix* é um dos exemplos mais completos de aplicação de sistemas de recomendação em larga escala. Em 2006, a empresa lançou o ***Netflix Prize***, competição que ofereceu US$ 1 milhão para a equipe que reduzisse em 10 % a raiz do erro quadrático médio (RMSE) na predição de avaliações de filmes em relação ao sistema ***Cinematch***. A competição estimulou milhares de equipes e permitiu que a empresa experimentasse novos algoritmos. A competição mostrou, entretanto, que há um limite ("*barreira mágica*") na precisão de predições de notas devido ao ruído natural nos dados de avaliações (Amatriain et al. 2015).

A experiência com o prêmio levou a *Netflix* a ampliar a visão sobre recomendação. A empresa percebeu que predições de nota não capturam todo o valor do sistema, a meta passou a ser maximizar o engajamento e a retenção de usuários. O serviço oferece um catálogo com milhares de vídeos e precisa ajudar as pessoas a encontrar conteúdos relevantes.

Para isso, a *Netflix* utiliza um **modelo centrado no usuário**, em que todos os itens têm custo semelhante, evitando favorecimento comercial e focando na satisfação do assinante. A personalização começa na página inicial, organizada em linhas horizontais como a **"*Top* 10"**, que apresenta os vídeos que um perfil (ou

família) provavelmente irá gostar. Há mecanismos como **"*Because you watched*"** para agrupar vídeos semelhantes a partir do histórico, e explicações de recomendação que mostram por que um título foi sugerido, aumentando a confiança do assinante.

Para sustentar a personalização, a *Netflix* coleta grandes volumes de dados – bilhões de avaliações e milhões de eventos diários de reprodução, buscas, inclusão em fila, etc. – além de metadados e *tags* sobre humor, qualidades e temas de cada vídeo. A infraestrutura é organizada em **três camadas** de computação (*off-line, nearline e on-line*), combinando processamento em lote, quase em tempo real e em tempo real para balancear latência e complexidade (Amatriain et al. 2015).

A *Netflix* utiliza um processo de **ciência de dados do consumidor** para avaliar novas ideias. As hipóteses são testadas em ciclos que combinam avaliações *off-line* e testes AB *on-line*. Métricas *off-line* incluem *RMSE*, ***normalised discounted cumulative gain* (*NDCG*)**, precisão, *recall*, cobertura e diversidade. Contudo, a métrica mais importante é a **retenção de assinantes**, medida via testes AB em que grupos de controle (versão atual) são comparados com grupos experimentais (nova abordagem). A empresa acompanha dezenas de métricas comportamentais (horas de visualização, taxa de cliques, seleção de recomendações) e só adota uma alteração se observar ganhos estatisticamente significativos na retenção, evitando comprometer a experiência com algoritmos muito lentos ou irrelevantes.

O caso da *Netflix* demonstra que sistemas de recomendação bem concebidos impactam diretamente métricas de negócios: Neil Hunt, diretor de produto, estimou que mais de 80 % dos filmes assistidos na plataforma em 2017 foram resultado de recomendações. O sucesso do serviço está ligado à capacidade de combinar dados ricos, arquitetura escalável e avaliação rigorosa para melhorar a experiência do usuário (Amatriain et al. 2015; Riedl et al. 1999).

4.2 Estudo de caso: Amazon

A *Amazon* foi uma das pioneiras na aplicação de sistemas de recomendação em *e-commerce*. Desde o final da década de 1990, a empresa vem personalizando sua loja de acordo com os interesses de cada cliente. Um elemento central dessa estratégia é o algoritmo de **filtragem colaborativa baseada em itens**. Smith et al 2017).

Diferentemente dos métodos usuais da época, que procuravam usuários com interesses semelhantes, a *Amazon* passou a identificar **itens relacionados** – ou seja, produtos comprados frequentemente em conjunto – e usa essa tabela de cocompra para gerar recomendações em tempo real. Esse algoritmo realiza grande parte do trabalho *offline* (construção da matriz de itens relacionados) e, no momento da recomendação, apenas consulta rapidamente essa estrutura para cada item do carrinho ou histórico do usuário. O método é escalável a centenas de milhões de usuários e itens e adapta-se instantaneamente quando um novo comportamento de compra é registrado.

O site oferece diversas funcionalidades baseadas nesse algoritmo:

- ***Customers who Bought***: em cada página de produto são exibidas listas de outros livros (ou autores) comprados por clientes que também adquiriram o item em questão. Isso permite sugerir produtos complementares, aumentar o tamanho médio do pedido e incentivar compras por impulso.
- ***Eyes***: serviço de notificação por e-mail que informa quando novos livros relacionados a autores, títulos, assuntos ou *ISBNs* escolhidos pelo cliente são adicionados ao catálogo. O usuário pode especificar critérios booleanos (AND/OR) e receber avisos persistentes.
- ***Amazon Delivers***: variação do *Eyes* em que o usuário seleciona categorias ou gêneros (biografias, culinária, etc.) e recebe periodicamente indicações de lançamentos.
- ***Book Matcher***: o cliente avalia livros de uma amostra em uma escala de 1 a 5. Após algumas avaliações, o sistema apresenta uma lista de obras que se correlacionam com o perfil indicado; o usuário fornece *feedback* ("*own it*", "*wish list*", "*not for me*") para refinar as sugestões.
- ***Customer Comments***: página de cada produto com avaliações e comentários textuais de outros clientes. Essas opiniões podem influenciar a decisão de compra (Smith et al. 2017; Barth. 2010).

Esses mecanismos têm objetivos distintos: converter visitantes em compradores ("***browsers into buyers***"), incentivar **vendas cruzadas (*cross-sell*)** ao sugerir itens complementares no carrinho e aumentar a **fidelidade**. O valor agregado vem do relacionamento criado com o cliente: quanto mais interage, mais preciso o modelo se torna, dificultando que o consumidor migre para concorrentes. Relatos indicam que uma proporção significativa de visualizações de páginas da *Amazon* é proveniente de recomendações; estudos de 2010 estimaram que cerca de 30 % das

páginas exibidas foram resultado de sistemas de recomendação (Riedl et al. 1999; Smith et al. 2017).

O impacto financeiro desses sistemas é considerável. A *Microsoft Research* estimou que 30 % do tráfego da página principal da *Amazon* provinha de recomendações e Neil Hunt, da *Netflix*, calculou que as recomendações valiam mais de US$ 1 bilhão por ano para a *Netflix*. A *Amazon* também reforça a importância da explicabilidade; o algoritmo baseado em itens pode justificar facilmente cada sugestão ("clientes que compraram este item também compraram…"), aumentando a confiança do usuário. (Riedl et al. 1999; Smith et al. 2017).

5. PROPOSTA DE SISTEMA DE RECOMENDAÇÃO PARA GESTÃO DE TAREFAS

Este capítulo apresenta o desenvolvimento da solução proposta, desde a definição do modelo conceitual do sistema de recomendação de tarefas voltado à gestão de equipes até sua implementação como prova de conceito. Inicialmente são descritos os componentes e fluxos que estruturam o modelo, relacionando fontes de dados, características das tarefas e perfis dos colaboradores.

Em seguida, detalha-se a arquitetura técnica adotada, que integra a plataforma *n8n* como arquitetura dos fluxos, o *PostgreSQL* como base de dados e agentes de Inteligência Artificial responsáveis e Código *Javascript* por ranquear e explicar as recomendações geradas. Por fim, são apresentados os testes funcionais realizados nos dois fluxos, destacando os pontos fortes, limitações e possibilidades de evolução da solução.

5.1 MODELO CONCEITUAL

Baseando-se nas lições dos casos da *Netflix* e da *Amazon*, propõe-se um **modelo conceitual** para um **sistema de recomendação de tarefas** voltado à gestão de equipes em *softwares* de produtividade. O objetivo é correlacionar demandas de atividades com as competências e a carga de trabalho de cada colaborador, sugerindo alocações que aumentem a produtividade e reduzam gargalos.

Coleta de dados:

- **Perfil de colaboradores** – informações sobre competências, experiência, certificações, preferências, disponibilidade e cargas atuais.

- **Características das tarefas** – descrição, complexidade, prazo, prioridade, dependências e requisitos de conhecimento.

- **Histórico de execução** – base de dados de tarefas concluídas, responsáveis, tempo de execução, *feedback* de qualidade e satisfação. Esse histórico permite treinar modelos de recomendação assim como as avaliações e ações de usuários alimentam os algoritmos da *Netflix*.

- **Sinais de atividade** – eventos de interação no *software* (início/fim de tarefas, trocas de status, comentários) em tempo real.

Processamento:

- **Pré-processamento e filtragem** – limpeza de dados, normalização de perfis e tarefas, e identificação de atributos importantes.

- **Algoritmo de recomendação** – sistema **híbrido** combinando filtragem colaborativa e abordagem baseada em conteúdo. A parte colaborativa identifica padrões de alocação a partir de tarefas concluídas por usuários de perfil semelhante; a parte de conteúdo avalia a correspondência entre as habilidades requeridas e as competências do colaborador.

- **Ranking e filtragem contextual** – as tarefas recomendadas são ranqueadas segundo probabilidade de sucesso e filtradas conforme carga de trabalho, prazo e prioridade. O modelo pode ponderar atributos diferentes, por exemplo, combinando disponibilidade e adequação com pesos aprendidos.

- **Interface e explicabilidade** – apresentar listas de tarefas com justificativas (por exemplo, "sugerida porque você concluiu tarefas de complexidade semelhante" ou "sugerida devido à sua competência em *React*"). A transparência estimula confiança, assim como a *Netflix* utiliza explicações para reforçar a adaptação às preferências.

5.1.1 Impacto esperado na produtividade

A integração de um sistema de recomendação de tarefas num *software* de produtividade tende a gerar impactos semelhantes aos observados na *Netflix* e na *Amazon* em seus domínios:

- **Redução da sobrecarga de informação** – assim como os sistemas de recomendação ajudam clientes a encontrar produtos entre milhões de opções, o algoritmo de tarefas filtra o *backlog* e propõe as atividades mais adequadas a cada colaborador.

- **Aumento do engajamento** – recomendações personalizadas e explicáveis podem aumentar a motivação e a satisfação; colaboradores sentem que suas

habilidades são reconhecidas e recebem desafios adequados (Amatriain et al 2015).

- **Melhor balanceamento de carga** – algoritmos consideram disponibilidade e histórico, distribuindo tarefas de forma equilibrada. Isso reduz gargalos e evita que alguns membros fiquem sobrecarregados, contribuindo para a eficiência global (Riedl et al. 1999).

- **Agilidade na tomada de decisão** – gestores passam a focar em atividades estratégicas enquanto o sistema sugere alocações iniciais; o tempo de atribuição diminui, aumentando a velocidade de execução de projetos (Smith et al 2017).

- **Aprimoramento contínuo** – ao coletar *feedback* e métricas de uso, o sistema aprende e adapta-se, melhorando sua precisão ao longo do tempo, de forma semelhante ao ciclo de experimentação da *Netflix*.

- **Maior transparência nas decisões** – ao apresentar não apenas a tarefa recomendada, mas também a justificativa e os critérios utilizados, o sistema torna o processo de alocação mais claro para gestores e colaboradores, reduzindo a sensação de arbitrariedade e aumentando a confiança nas decisões do time.

Em síntese, os estudos de caso de *Netflix* e *Amazon* mostram que sistemas de recomendação bem avaliados podem transformar produtos e serviços, aumentando engajamento, vendas e retenção.

Ao adaptar essas práticas a um contexto de **gestão de tarefas**, é possível construir um modelo conceitual que utiliza dados de competências e histórico para sugerir alocações de forma inteligente, apoiado por métricas de avaliação robustas. Este modelo tende a aumentar a produtividade ao reduzir o esforço manual de alocação, otimizar o uso de habilidades da equipe e acelerar a realização de tarefas

5.2 DESENVOLVIMENTO DA SOLUÇÃO

Esta seção descreve o processo de desenvolvimento da solução proposta, com foco na arquitetura do sistema de recomendação, na coleta e preparação dos dados e no delineamento das etapas futuras de testes e validação. A abordagem baseia-se em integrar um fluxo de automação via *n8n* a um agente de IA que utiliza o modelo *ChatGPT* para analisar e recomendar tarefas de forma inteligente.

5.2.1 Infraestrutura do Sistema de Recomendação: n8n e PostgreSQL

n8n é uma plataforma *low-code* de orquestração de fluxos que permite compor integrações e rotinas de dados por meio de nós visuais e trechos de código quando necessário.

No contexto deste trabalho, o *n8n* faz o papel de Arquitetura do sistema de recomendação: conecta a *API* do sistema, executa a preparação/limpeza dos dados, chama o agente de IA para *reranking* e, por fim, valida e formata a saída. Essa separação entre orquestração, inferência e validação segue práticas adotadas em *pipelines* industriais de recomendação, nas quais há camadas distintas para coleta/*feature store*, *serving online* e experimentação (Amatriain et al 2015).

Já o *PGVector* é uma extensão do *PostgreSQL* que adiciona o tipo vector e índices vetoriais para busca por similaridade.

Ele viabiliza um **"banco vetorial"** dentro do próprio *PostgreSQL*, permitindo armazenar *embeddings* (representações numéricas) de tarefas e consultá-las por proximidade semântica. Na prática, isso oferece duas vantagens ao Sistema de Recomendação:

- **Contexto e memória** — recuperar "tarefas parecidas" ou "casos históricos" durante o *reranking*, enriquecendo as decisões do agente;
- **Baixa latência** — servir consultas vetoriais próximas ao dado transacional, sem manter uma infraestrutura separada.

Essa estratégia é coerente com abordagens modernas de Sistemas de Recomendação que usam representações vetoriais para comparar itens/usuários e apoiar recomendações personalizadas (Riedl et al 1999).

5.2.2 Arquitetura do Sistema de Recomendação

O sistema de recomendação de tarefas foi concebido como dois fluxos de automação. Sendo o primeiro fluxo:

Composto por três componentes principais: (1) um **nó *HTTP*** responsável por consultar a *API* do *software* de gestão de tarefas, (2) um **agente de IA** que processa os dados de tarefas e o perfil do usuário utilizando o modelo *ChatGPT*, e (3) nós de **código *Javascript*** que validam, ranqueiam e filtram os dados.

A arquitetura foi implementada na plataforma *n8n*, que permite conectar serviços por meio de nós visuais.

Visão Geral do primeiro fluxo (Figura 1)

Os nós que compõem a Arquitetura são:

- *Get -> Tarefas Neotriad – Tipo: HTTP Request – Papel Principal: Coleta da API*
- *Retorna todas as Tarefas – Tipo: Code – Papel Principal: Simplifica e filtra os dados*
- *Prepara os dados das tarefas – Tipo: Code – Papel Principal: Normaliza e calcula heuristica*
- *AI Agent – Tipo: LLM Agent – Papel Principal: Ranqueia as tarefas e guarda no Banco*
- *Postgres PGVector Store – Tipo: Vector Store – Papel Principal: Guarda tarefas no banco pra busca semântica*
- *Combina Dados – Tipo: Merge – Papel Principal: Junta entradas pra validação*
- *Valida dados LLM – Tipo: Code – Papel Principal: Valida dados e volta os dados das principais tarefas com base no score do usuário*

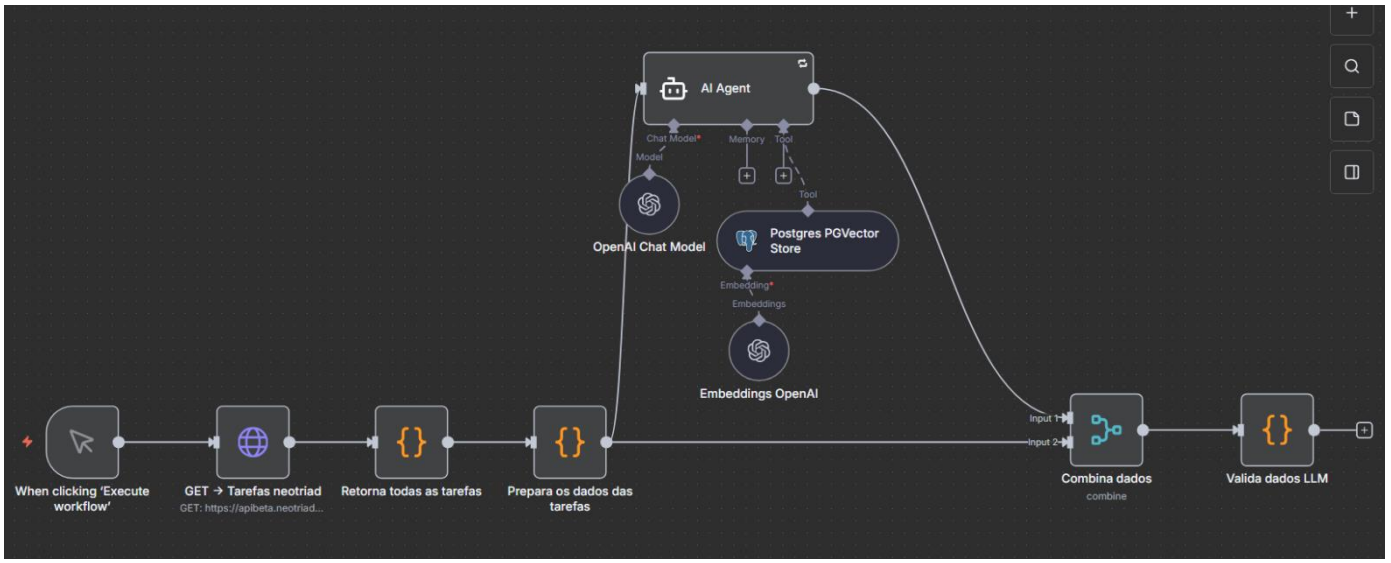

Figura 1 - Primeiro fluxo no *n8n*

O nó *HTTP* pode ser usado como uma ferramenta dentro de um agente de IA. Ele funciona de forma semelhante ao nó padrão, mas é voltado à coleta de informações de *websites* ou *APIs* quando acoplado a um agente. Isso permite que o agente utilize dados externos como entrada para suas análises. A integração com o modelo da *OpenAI* ocorre por meio de um conector que envia o conteúdo das tarefas e recebe como resposta sugestões de alocação, explicações ou enriquecimento dos dados.

Coleta e Preparação dos Dados

A coleta de dados inicia-se com uma chamada ***HTTP GET*** ao *endpoint* do sistema de gestão (API Neotriad). O método *GET* é apropriado para recuperar informações, pois solicita apenas a representação de um recurso; os parâmetros são passados na *URL* por meio de pares nome/valor. Na implementação, os parâmetros início e termino definem o intervalo das tarefas a serem retornadas. A *URL* utilizada é:

GET https://apibeta.neotriad.com/api/tarefas?inicio=YYYY-MM-DD&termino=YYYY-MM-DD

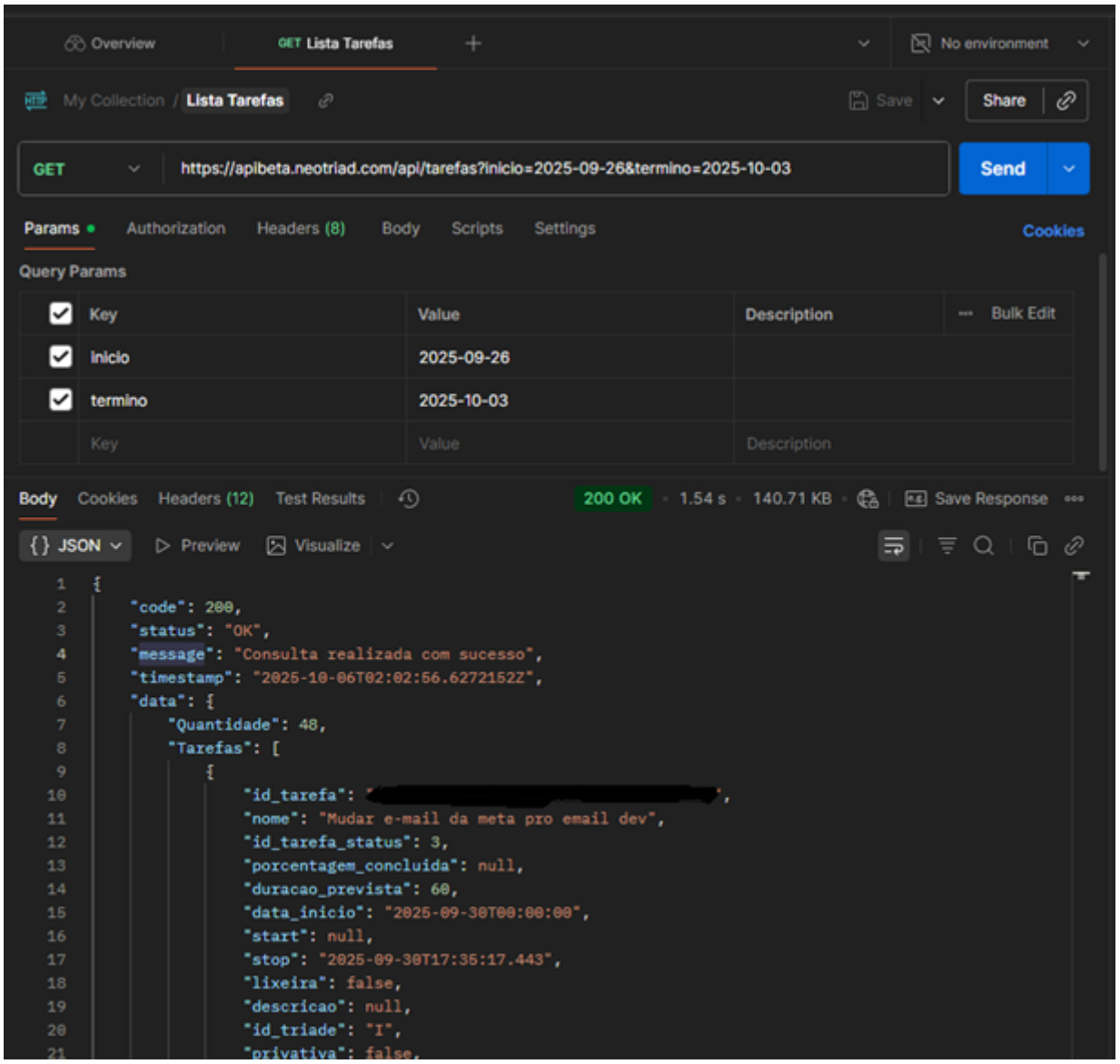

Figura 2 - Exemplo de saída *http* usando *Postman*

Figura 3 - Exemplo de *http* dentro do *n8n*

Após a obtenção, os dados passam por uma etapa de **preparação**. Nessa fase, os registros são filtrados para remover tarefas duplicadas ou tarefas que não estejam no escopo da recomendação (por exemplo, itens marcados como lixeira ou tarefas concluídas). Em seguida:

Nó “Retorna todas as tarefas” (Figura 4)

- **Normalização dos campos** – converte-se datas e horários para um formato padrão e transforma-se o status da tarefa em rótulos interpretáveis pelo modelo (por exemplo, pendente, em andamento, concluída).
- **Enriquecimento com perfil do usuário** – associa-se cada tarefa às competências e disponibilidade do colaborador a partir do perfil descrito no Capítulo 3; essa associação é essencial para avaliar a adequação da tarefa ao usuário.

Nó “Prepara dados das tarefas” (Figura 4)

- **Criação de atributos derivados** – calcula-se a duração estimada (diferença entre *start* e *stop*), determina-se o tempo restante até o prazo e identifica-se a prioridade com base na combinação de status e proximidade do prazo.

Figura 4 - Nós de código *Javascript* dentro do n8n

Nós “*Ai Agent*” e “*Postgres PGVector Store*” (Figura 5)

- **Estruturação em formato vetorial** – organiza-se cada tarefa e perfil em vetores de características para alimentar os algoritmos de recomendação (fatoração de matrizes e modelos híbridos).
- **Ranqueamento com base nos dados especificos** – Os dados são enviados para um Agente de IA, que com base no histórico e regras de negócio, ranqueia e retorna as 10 “melhores” tarefas com título, *score* e “*rationale*” que seria o motivo da escolha

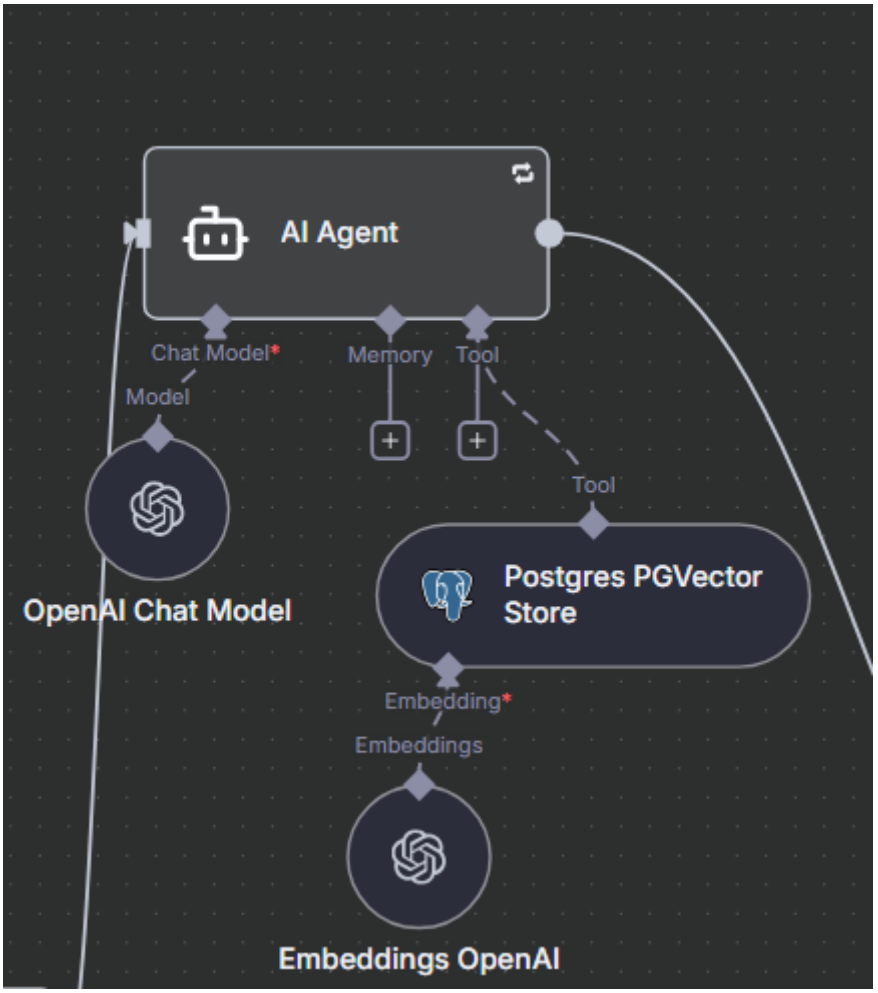

Figura 5 - Nós de Agente de IA e banco vetorial no *n8n*

Após fase de preparação, é feito uma validação dos dados originais com os dados do Agente de IA, onde eles são combinados em um único objeto, onde permite que o nó “Valida dados *LLM*” transforme a saída da sugestão da IA em uma saída padronizada e garantindo a integridade dos dados (Figura 6)

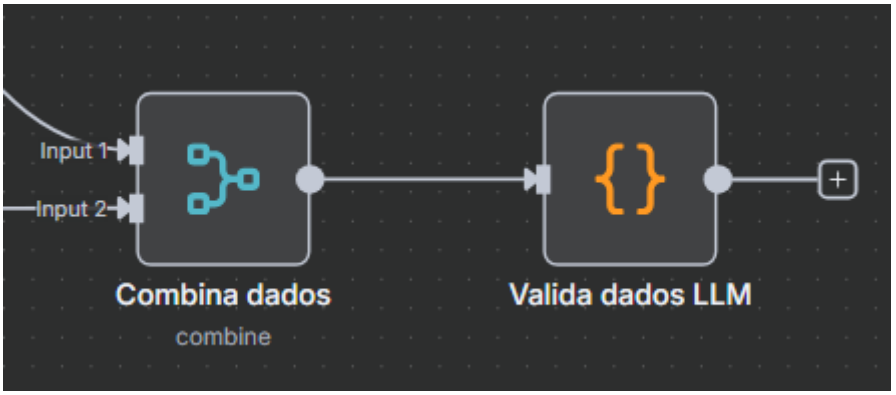

Figura 6 - Nós de *Merge* e Código *Javascript* do *n8n*

Explicação dos dados:

- *Task_id: id da task do usuário na plataforma*
- *Title: título da task*
- *Triad: Importância da tarefa: I – Importante, C – Circunstancial, U – Urgente*
- *Duration_min: duração em minutos*
- *Score: Prioridade da task*
- *Rationale: Justificativa do porque recebeu o score*

O Segundo fluxo complementa o primeiro, enquanto o primeiro produz o ranqueamento das tarefas específicas pro usuário individualmente, o segundo permite consultar o sistema via *chat*, recuperar o histórico vetorial das tarefas dos colaboradores e alocar a melhor pessoa pra cada tarefa, mostrando *score* (0-1) e *rationale* (justificativa) por tarefa.

Ele segue o padrão de recomendação em duas etapas: Recuperação dos dados já cadastrados e Ranqueamento desses dados, pra produzir uma lista ordenada com justificativas, o que melhora a aceitação e a confiança do sistema pelo usuário (Amatriain et al 2015).

Nós que compõem a arquitetura (Figura 7)

- *Chat – Tipo: Chat Trigger – Papel Principal: iniciar a execução a partir de uma pergunta*
- *AI Agent – Carrega Tarefas – Tipo: LLM Agent – Papel Principal: carrega do banco vetorial os perfis vetorizados dos colaboradores e suas tarefas*
- *Dados de score tarefas de usuários – Tipo: Code – Recebe os dados dos usuários que estão no banco*
- *Formata e normaliza score de tarefas – Tipo: Code – Formata, organiza e normaliza os campos de tarefas*
- *Ai Agent – Resposta Natural – Tipo: LLM Agent – Faz a busca semântica com base nos dados e propõe a resposta final*
- *Salvar a resposta no banco – Tipo: PostgreSQL – salva a tarefa e a resposta no banco de dados, pra uma futura análise do sistema.*

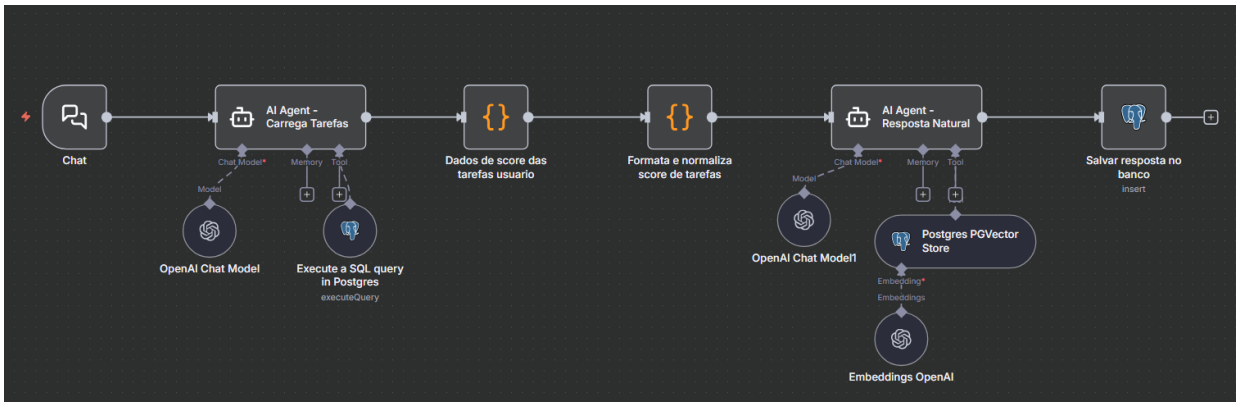

Figura 7 - Arquitetura do segundo fluxo no *n8n*

Visão geral do segundo fluxo

A interação começa no nó "*Chat*", onde o **gestor** pode escrever diretamente a tarefa no chat, onde o fluxo vai iniciar e rodar a automação. (Figura 7)

A tarefa em si pode ser escrita de maneira direta, como:

- "Quem seria a pessoa ideal pra fazer a tarefa: arrumar *bug* do sistema *mobile*, Importante(tríade) com 120 minutos"

O Agente de IA tem uma engenharia de *prompt* pra ser mais específica ou se o usuário digitar errado, como por exemplo pegar somente dois usuários:

- "Entre o colaborador X e Y, quem seria ideal pra fazer a tarefa: Arrumar *bug* do sistema *web*, urgente, 60 minutos"

Após a mensagem chegar ao nó "*AI Agent* – Carrega Tarefas" é executado uma *Query SQL* que é feita de forma dinâmica, ou seja, esse agente pega todos os dados dos colaboradores no banco, ou somente de alguns dependendo da pergunta do usuário. (Figura 8)

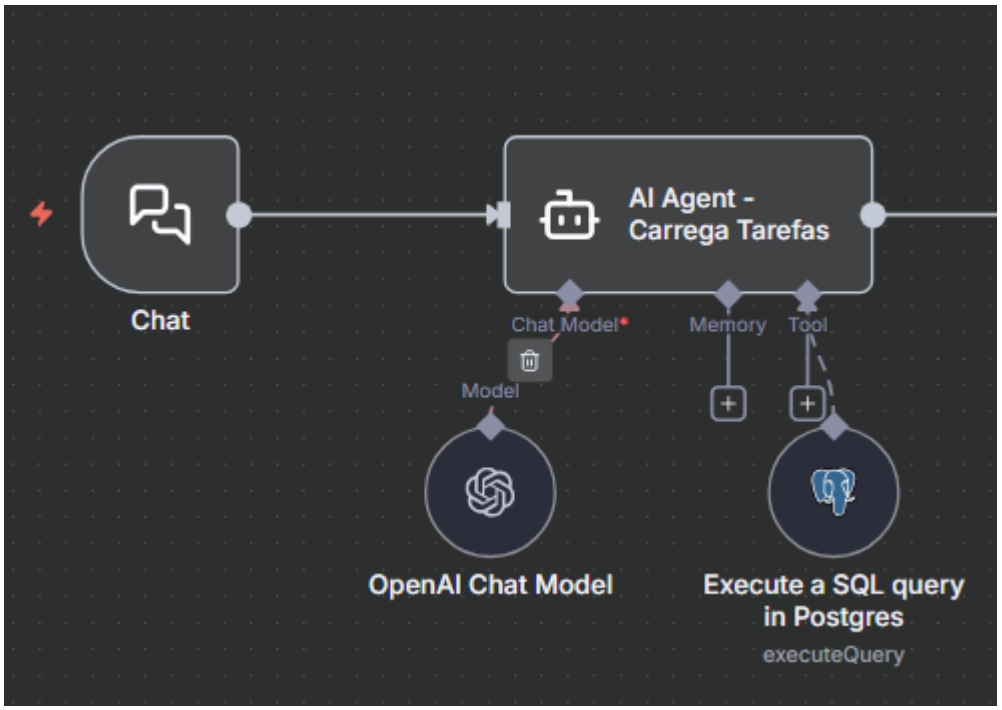

Figura 8 - Nós de chat trigger + Agente IA **n8n**

Após pegar esses dados, eles são normalizados, formatados e ordenados pelo histórico de cada pessoa, e a sua similaridade com a tarefa executada pelos dois nós da figura. (Figura 9)

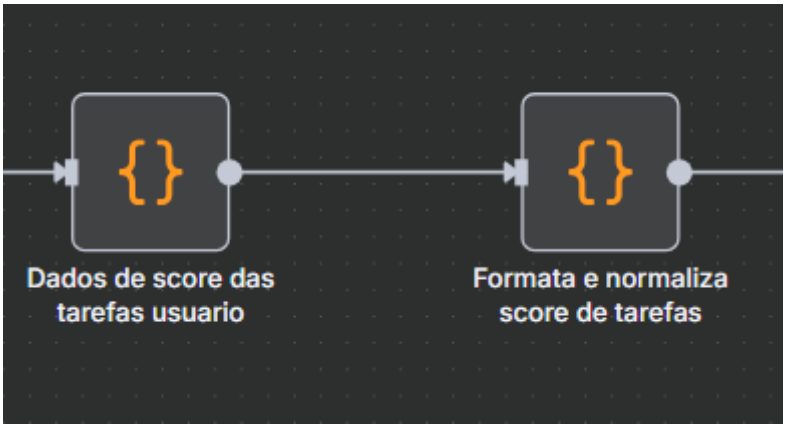

Figura 9 - Nós de Código *Javascript* segundo fluxo n8n

O Nó "*AI Agent* – Resposta Natural" transforma os dados estruturados vindo dos nós anteriores e com base nas métricas e regras de negócios adicionadas nos *prompts*, transforma a resposta em uma decisão explicada e comunicável, onde: Seleciona o melhor colaborador pra tarefa, atribui um *score* (0-1) e gera um *rationale* (frase justificando o porquê essa pessoa foi escolhida)

E no final salva as respostas no banco de dados *PostgreSQL*, pra uma futura análise das respostas e validação do modelo (Figura 10)

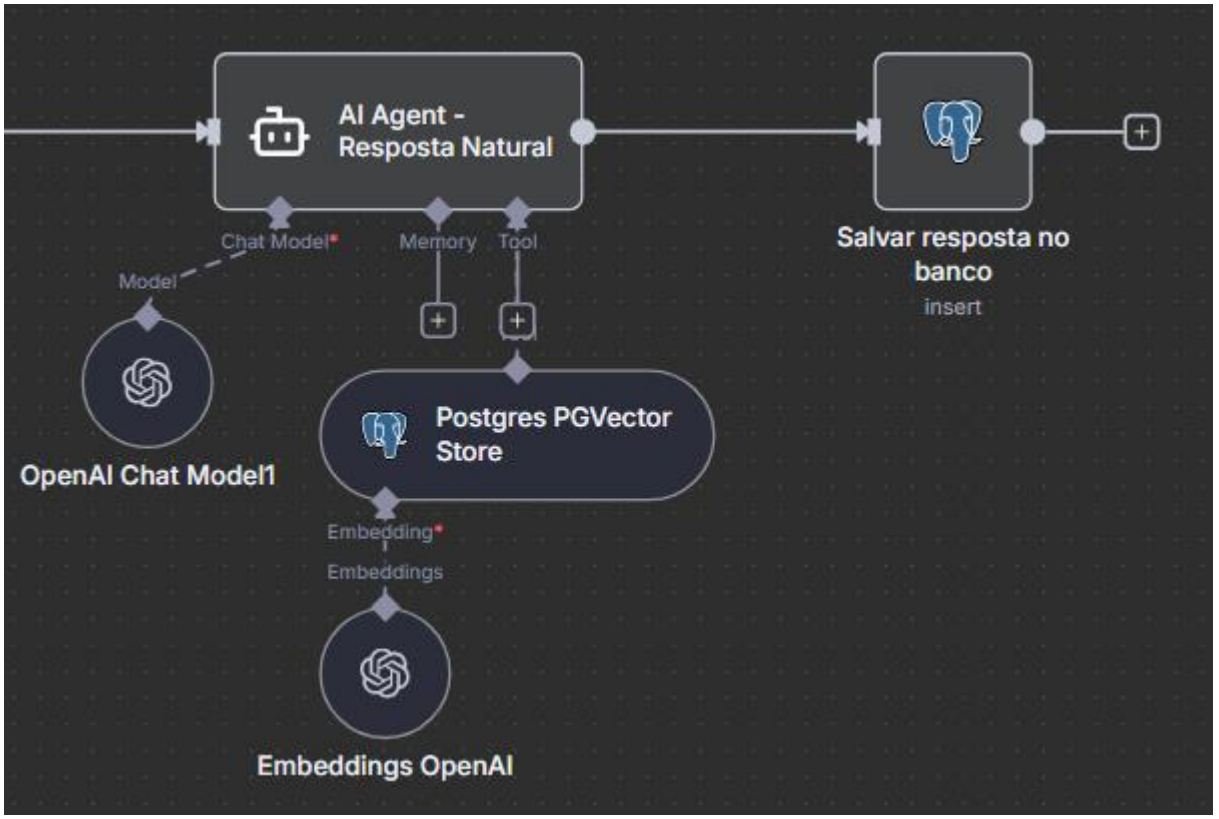

Figura 10 - Nós de Agente de IA + PostgreSQL segundo fluxo n8n

O Exemplo de duas saídas com duas perguntas diferentes:

- “Quem seria o melhor colaborador pra resolver a tarefa: “Arrumar modelo de resposta do *whatsapp* da *Meta*, urgente”.

Resposta: “A pessoa mais adequada pra essa tarefa é o Maicon com *score* 0.92, porque apresenta similaridade e histórico com ajustem em respostas do *Whatsapp/Meta* e bom desempenho em tarefas urgentes”

- “Quem seria o melhor colaborador pra resolver a tarefa: Arrumar problema no assistente imobilizado sobre mensagem específica do cliente, urgente, 20 minutos”.

Resposta: “A pessoa mais adequada pra essa tarefa é o Eric com *score* 0.89, porque mostra alta similaridade com tarefas sobre assistente de imobilizado e com tarefas de durações curtas”

5.3 Testes do Sistema de Recomendação

Esta seção descreve testes funcionais simples para verificar se os dois fluxos do Sistema de Recomendação estão entregando o esperado: formato correto, ordenação coerente e resposta clara com *score* e *rationale*.

5.3.1 Casos de Teste – Fluxo 1

CT1 – Teste básico – Colaborador 1

Entrada: *Acess_token* do colaborador no *HTTP Request*

Saída Esperada: *JSON* ordenado pelo *score*, com título, duração, tríade e *rationale*

Saídas dos testes:

Formato JSON

[

{

"task_id": " Id da tarefa ",

"title": "mudar valor default criar tarefas",

"triad": "I",

"duration_min": 5,

"score": 1,

"rationale": "I e 5min — ganho rápido e impacto direto; priorizado na ausência de U."

},

{

"task_id": " Id da tarefa ",

"title": "Organizar meu dia",

"triad": "I",

"duration_min": 10,

"score": 0.97,

"rationale": "I e 10min — curta e orienta o restante do dia; alto retorno."

},

{

"task_id": " Id da tarefa ",

"title": "Ver Emails/Teams/Contatos manhã",

"triad": "I",

"duration_min": 5,

"score": 0.95,

"rationale": "I e 5min — verificação rápida da manhã; mantido alto, evitando empilhar tarefas similares no topo."

}

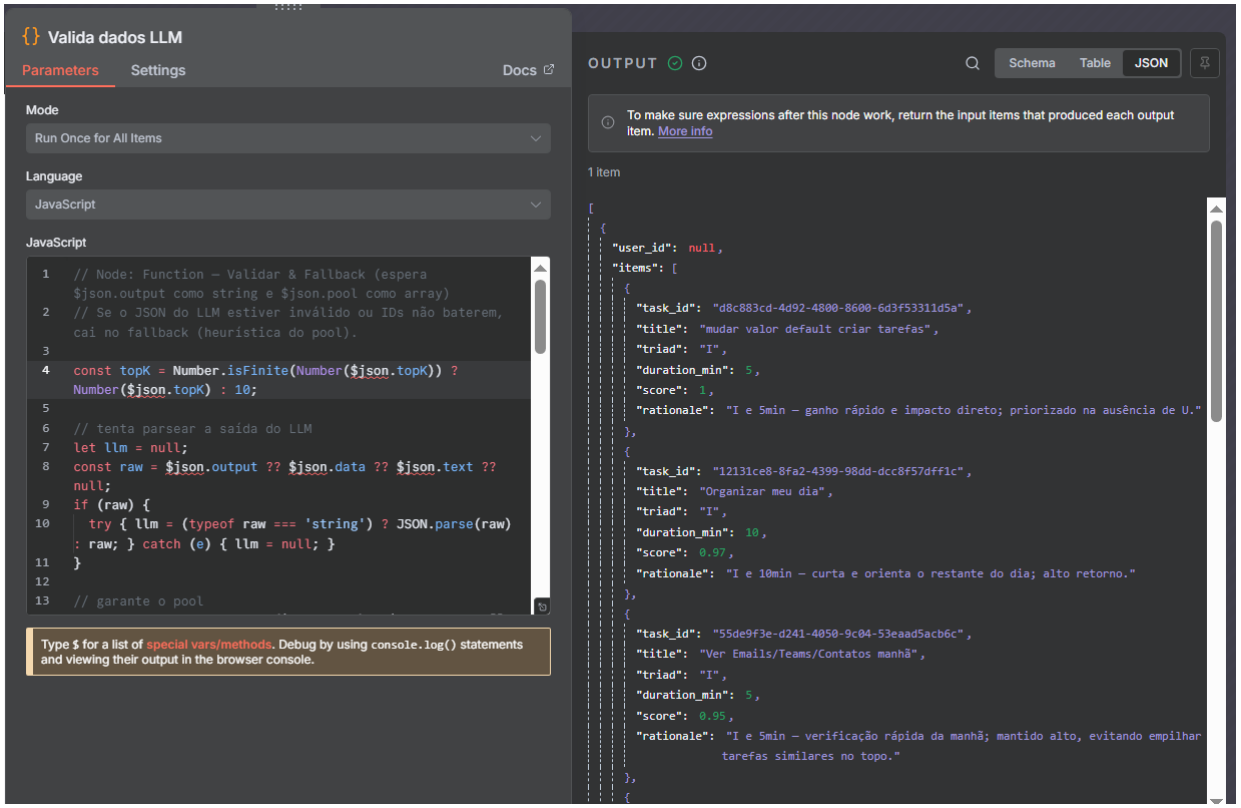

Figura 11 - Exemplo de saída teste CT1 n8n

CT2 – Teste básico – Colaborador 2

Entrada: *Acess_token* do colaborador no *HTTP Resquest*

Saída Esperada: *JSON* ordenado pelo *score*, com título, duração, tríade e *rationale*

Resultado Obtido:

Formato JSON

[

{

"task_id": " Id da tarefa ",

"title": "🐞 Relatório Lista - Não abre lista da Equipe (se abrir pela aba \"relatórios\")",
"triad": "U",
"duration_min": 60,
"score": 0.93,
"rationale": "Triad U (urgente) > I/C. Mesmo com 60 min, corrige bug crítico e destrava; foco máximo."
},
{
"task_id": "Id da tarefa",
"title": "Buscar faturas da Sabesp (mãe)",
"triad": "I",
"duration_min": 0,
"score": 0.9,
"rationale": "Triad I; duração 0 min (quick win) gera tração imediata. Sem info de prazo, leve ajuste no score."
},
{
"task_id": " Id da tarefa ",
"title": "📝 Mapear Infra da sync Outlook / Google (Dia 3)",
"triad": "I",
"duration_min": 5,
"score": 0.889,
"rationale": "Triad I; 5 min. Mapeamento técnico curto que destrava próximos passos."
}

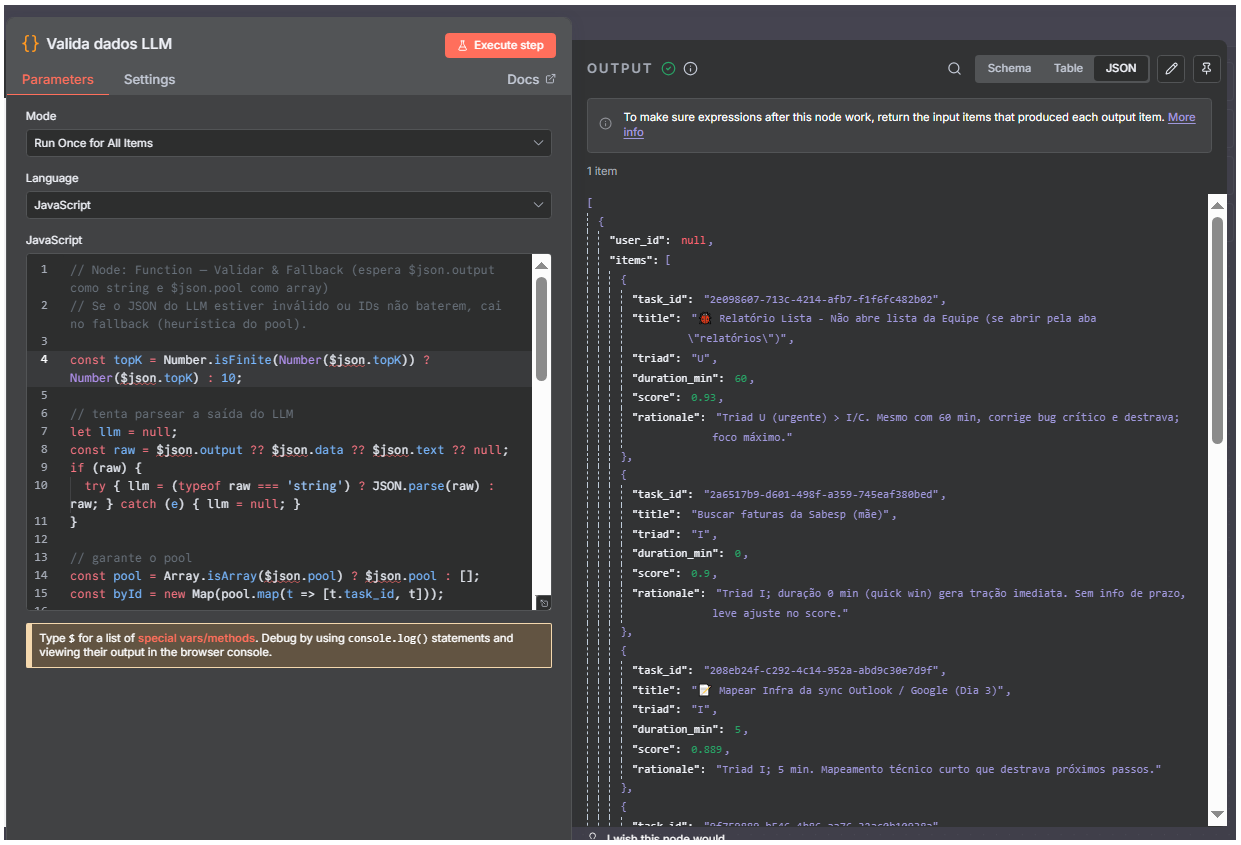

Figura 12 - Exemplo de saída teste CT2 n8n

5.3.2 Casos de Teste – Fluxo 2

CT3 – Pergunta livre por texto

Entrada: “Quem seria o melhor colaborador pra resolver a tarefa: “Arrumar modelo de resposta do whatsapp da *Meta*, urgente”

Saída Esperada: Frase no padrão “O melhor colaborador é o colaborador x com nota xx porque (justificativa)”

Resultado obtido:” A pessoa mais adequada pra essa tarefa é o Maicon com *score* 0.92, porque apresenta similaridade e histórico com ajustem em respostas do Whatsapp/Meta e bom desempenho em tarefas urgentes”

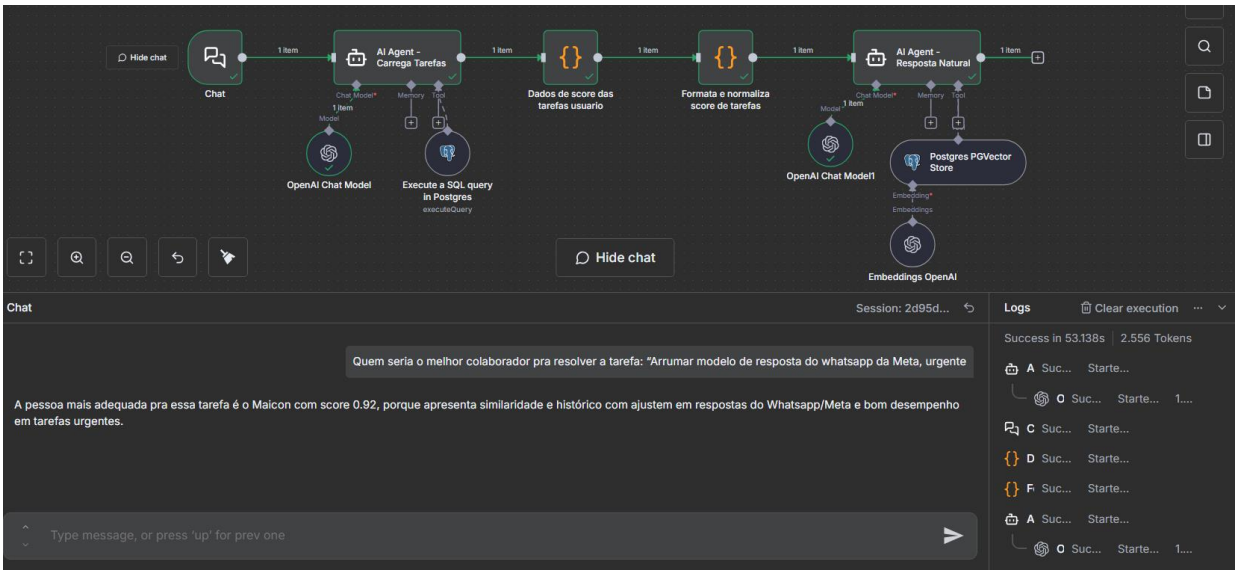

Figura 13 - Exemplo de saída teste CT3 n8n

CT4 – Pergunta livre por texto

Entrada: "Quem seria o melhor colaborador pra resolver a tarefa: Arrumar problema no assistente imobilizado sobre mensagem específica do cliente, urgente, 20 minutos"

Saída Esperada: Frase no padrão "O melhor colaborador é o colaborador x com nota xx porque (justificativa)"

Resultado obtido:" A pessoa mais adequada pra essa tarefa é o Eric com *score* 0.89, porque mostra alta similaridade com tarefas sobre assistente de imobilizado e com tarefas de durações curtas"

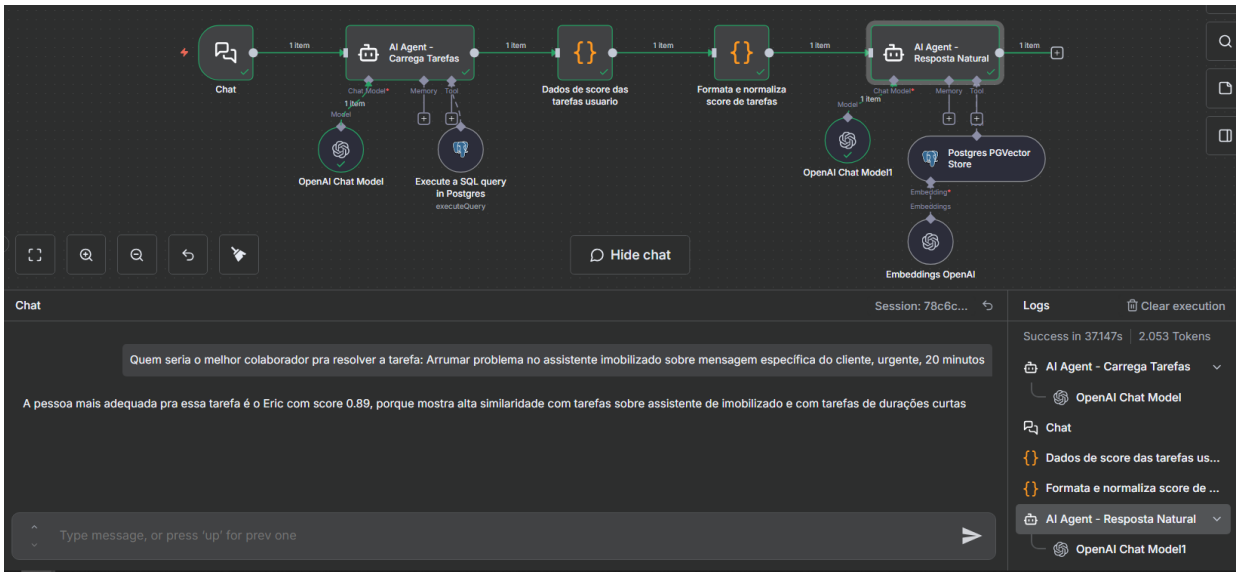

Figura 14 - Exemplo de saída teste CT4 n8n

CT5 – Comparação específica entre dois colaboradores

Entrada: “Entre o Maicon e o Eric, quem seria melhor pra fazer a tarefa: Arrumar *bugs* do agente Whatsapp da Neotriad, importante, 120 minutos”

Saída Esperada: Frase no padrão “Entre os xxx Colaboradores, o que teve a maior *score* foi o colaborador xxx com xx, seguido por xxxx”

Resultado obtido:” Entre os dois colaboradores, o que teve a maior nota foi o Maicon com 0.95, seguido pelo Eric com 0.33. Isso se deve ao fato de o Maicon ter mais tarefas concluídas e histórico com tarefas do assunto especificado”

Figura 15 - Exemplo de saida teste CT5 n8n

CT6 – Tarefa muito fora do escopo

Entrada: “Quem seria o melhor colaborador pra resolver a tarefa: Jogar bola no sábado, urgente”

Saída Esperada: Frase no padrão “Nenhum dos colaboradores passou da nota 0.5 pra essa tarefa, sem candidatos”

Resultado obtido:” Sem candidatos pra essa tarefa”

Figura 16 - Exemplo da saída teste CT6 n8n

5.4 Análise da Solução

A Solução combina dois fluxos: O primeiro focado em Priorização da tarefa por colaborador e o segundo em Alocação de tarefas por colaborador via *chat* usando histórico vetorial. Essa arquitetura em duas etapas (recuperação + ranqueamento) é recomendada por separar Orquestração, Inferência e Validação (Amatriain et al 2015).

O que funcionou bem:

- Formato e ordem das saídas: Os fluxos 1 e 2 retornam *JSON*, *score* e *rationale* padronizados.
- Explicabilidade: Os *scores*, *rationales* e respostas dos algoritmos são curtos, diretos e específicos.
- Memória e Contexto: O banco vetorial permite recuperar os dados dos colaboradores de forma prática e organizada.
- Transparência: Registro de Entrada/Saída e banco de dados viabilizam análises, testes e melhorias futuras
- Arquitetura bem definida: *n8n* sendo a Arquitetura, Agentes de IA pra ranqueamento e Banco vetorial pra contexto.

Limitações do sistema:

- Cobertura de histórico: Pouco histórico pra colaboradores podem reduzir a qualidade da saída e manipular os dados

- Dependência de texto: Depende muito de o usuário digitar um texto bem escrito e bem definido pra uma boa saída
- Sensibilidade ao *prompt*: Dependendo do modelo usado, as respostas podem variar.
- Custo não medido: Não tem avaliação de custo nem tempo e resposta

6. CONCLUSÃO

O presente trabalho teve como objetivo propor um modelo de Sistema de Recomendação de tarefas voltado à gestão de equipes e produtividade, investigando como técnicas de Inteligência Artificial podem apoiar a distribuição de atividades de forma mais eficiente e alinhada ao perfil dos colaboradores. Ao longo da pesquisa, foram discutidos os fundamentos de sistemas de recomendação, conceitos de produtividade, gestão de equipes e gestão de projetos, além de se analisar como grandes empresas, como *Netflix* e *Amazon*, estruturam seus sistemas para lidar com grandes volumes de opções e decisões diárias.

A revisão de literatura evidenciou que sistemas de recomendação, quando bem projetados, podem aumentar a satisfação dos usuários, reduzir a sobrecarga de informação e apoiar decisões em ambientes complexos, não apenas em contextos de consumo, mas também em cenários corporativos. A partir dessa base teórica, foram explorados conceitos de gestão de tempo, priorização e fluxo de trabalho, mostrando como a combinação entre dados históricos, competências, disponibilidade e prioridades de negócio pode gerar recomendações mais justas e transparentes para a equipe.

Com base nas lições extraídas dos estudos de caso da *Netflix* e da *Amazon*, foi definido um modelo conceitual de sistema de recomendação de tarefas voltado à gestão de equipes, especificando fontes de dados, atributos de tarefas e colaboradores, etapas de processamento e critérios de ranqueamento e explicabilidade. Em seguida, esse modelo foi traduzido em uma prova de conceito integrada a uma ferramenta de gestão de tarefas, utilizando fluxos de automação, banco de dados vetorial e agentes de IA para recuperar, tratar e ordenar tarefas de acordo com o contexto de cada usuário. Dessa forma, foi possível demonstrar, ainda que em ambiente controlado, como um gestor pode ser apoiado por recomendações que consideram tanto o histórico de execução quanto a carga atual de trabalho e a importância de cada atividade.

Conclui-se que sistemas de recomendação de tarefas baseados em Inteligência Artificial representam uma abordagem promissora para apoiar a gestão de equipes e a organização do trabalho em ambientes corporativos. A combinação

entre fundamentos teóricos consolidados, estudo de casos de referência na indústria e a implementação de uma prova de conceito mostrou que é viável adaptar técnicas utilizadas em grandes plataformas de consumo para o contexto de produtividade e distribuição de tarefas. Embora ainda haja desafios relacionados à qualidade dos dados, avaliação em larga escala e adoção pelas organizações, este trabalho contribui ao apresentar um modelo conceitual e uma implementação inicial que podem servir de base para soluções futuras, reforçando o papel da IA como aliada dos gestores e das equipes na busca por mais eficiência, transparência e equilíbrio na gestão do trabalho.

REFERÊNCIAS

AMATRIAIN, Xavier et al. Recommender systems in industry: a Netflix case study. 2015. Disponível em: https://link.springer.com/chapter/10.1007/978-1-4899-7637-6_11. Acesso em: 04 set. 2025.

ANTONIO, Gestão de Projetos de software. 2009. Disponível em: https://www.devmedia.com.br/gestao-de-projetos-de-software/9143# Acesso em: 05 dez. 2025.

BARTH, Fabrício J. Modelando o perfil do usuário para a construção de sistemas de recomendação: um estudo teórico e estado da arte. 2010. Disponível em: https://fbarth.net.br/docs/bar2010a.pdf. Acesso em: 04 set. 2025.

BRUNIALTI, Lucas F. et al. Aprendizado de máquina em sistemas de recomendação baseados em conteúdo textual: uma revisão sistemática. 2015. Disponível em: https://sol.sbc.org.br/index.php/sbsi/article/view/5818. Acesso em: 04 set. 2025.

JIANG, Liaoliang et al. A trust-based collaborative filtering algorithm for e-commerce recommendation system. 2018. Disponível em: https://link.springer.com/content/pdf/10.1007/s12652-018-0928-7.pdf. Acesso em: 03 set. 2025.

KNIJNENBURG, Bart P. et al. Explaining the user experience of recommender systems. 2012. Disponível em: https://link.springer.com/content/pdf/10.1007/s11257-011-9118-4.pdf. Acesso em: 03 set. 2025.

PAULA, Cátia Sirlene Martins de. O papel da gestão de equipes e sua relevância no processo de desenvolvimento de projetos. 2023. Disponível em: https://revista.fatectq.edu.br/interfacetecnologica/pt_BR/article/view/1624/922. Acesso em: 05 dez. 2025.

PU, Pearl et al. A user-centric evaluation framework of recommender systems. 2011. Disponível em: http://www.comp.hkbu.edu.hk/~lichen/download/paper3.pdf. Acesso em: 03 set. 2025.

PU, Pearl et al. Evaluating recommender systems from the user's perspective: survey of the state of the art. 2012. Disponível em: https://doc.rero.ch/record/317166/files/11257_2011_Article_9115.pdf. Acesso em: 03 set. 2025.

RIEDL, John et al. Recommender systems in e-commerce. 1999. Disponível em: https://dl.acm.org/doi/pdf/10.1145/336992.337035. Acesso em: 04 set. 2025.

RIEDL, John et al. Is seeing believing? How recommender interfaces affect users' opinions. 2003. Disponível em: https://www.researchgate.net/profile/John-Tsibouklis/publication/200110346_Is_seeing_believing_how_recommender_system_interfaces_affect_users'_opinions/links/0c96053b4f7a29c7fb000000/Is-seeing-believing-how-recommender-system-interfaces-affect-users-opinions.pdf. Acesso em: 03 set. 2025.

SMITH, Brent et al. Two decades of recommender systems at Amazon.com. 2017.

Disponível em: https://www.amazon.science/publications/two-decades-of-recommender-systems-at-amazon-com. Acesso em: 04 set. 2025.